

Comprehensive Fourier Series Practice (Part 1)

Test Preparation Guide

Prerequisite: The Testing Utility

For all the following questions, assume you have already loaded the signal and instantiated the original object. We will also define a standard Mean Squared Error (MSE) function that will be used repeatedly.

```
import numpy as np
import matplotlib.pyplot as plt
from fs_redrawer import FourierEpicycles
from svg_utils import load_svg_path

# Load original signal
t, z = load_svg_path('svgs/heart.svg', num_points=1000)
N_HARMONICS = 50
fs_orig = FourierEpicycles(t, z, N_HARMONICS)
fs_orig.calculate_all_coefficients()

def calculate_mse(array1, array2):
    """Computes the Mean Squared Error between two complex arrays."""
    return np.mean(np.abs(np.array(array1) - np.array(array2))**2)
```

1 Level 1: Basic Implementation (Single Properties)

1.1 Question 1: Time Shifting

Problem Statement: Shift the drawing in time by $t_0 = T/4$. Compute the Fourier coefficients of the shifted signal using your OOP framework. Verify the Time Shifting property by comparing your numerical coefficients with the theoretical prediction. Compute the MSE for both magnitude and phase.

Theory & Intuition: The Time Shifting property states that if $z(t) \leftrightarrow c_n$, then:

$$z(t - t_0) \leftrightarrow c_n e^{-jn\omega t_0} \quad (1)$$

Intuition: Delaying a signal does not change its shape (magnitude of coefficients remains the same), but it rotates the starting angle (phase) of every epicycle. Higher frequency epicycles must rotate more to achieve the same time delay, hence the n in the phase shift.

Code Solution:

```
# 1. Create the shifted signal
# Since t spans [0, T] with 1000 points, shifting by T/4 means rolling the array by 1000/4 = 250 indices.
shift_idx = len(z) // 4
z_shifted = np.roll(z, shift_idx)

# 2. Compute numerical coefficients using OOP framework
fs_shifted = FourierEpicycles(t, z_shifted, N_HARMONICS)
fs_shifted.calculate_all_coefficients()

# 3. Theoretical prediction and Verification
t0 = fs_orig.T / 4
mse_mag_list = []
mse_phase_list = []

for n in range(-N_HARMONICS, N_HARMONICS + 1):
    c_n_orig = fs_orig.coeffs[n]
    c_n_num = fs_shifted.coeffs[n] # Numerical

    # Theoretical: c_n * e^{-j n omega t_0}
    c_n_theo = c_n_orig * np.exp(-1j * n * fs_orig.omega * t0)

    mse_mag_list.append(np.abs(np.abs(c_n_num) - np.abs(c_n_theo))**2)
    # Phase difference needs careful unwrapping, but we can just compare the complex numbers directly for overall MSE
    mse_phase_list.append(np.abs(np.angle(c_n_num) - np.angle(c_n_theo))**2)

print(f"Time Shift Mag MSE: {np.mean(mse_mag_list)}")
print(f"Time Shift Phase MSE: {np.mean(mse_phase_list)}")
```

Code Explanation & Alternatives: We used `np.roll` to shift the discrete array. This works perfectly because the assignment explicitly states the signal is periodic and wraps around ($z[-1] == z[0]$). *Alternative Method for Time Shifting:* If t_0 was not a perfect integer multiple of the sample spacing, `np.roll` would fail. The alternative is to use `np.interp` to resample the signal:

```
# Alternative Time Shift (Continuous interpolation)
t_shifted = (t - t0) % fs_orig.T
# np.interp only works on real arrays, so we split it
z_shifted_alt = np.interp(t_shifted, t, z.real) + 1j * np.interp(t_shifted, t, z.imag)
```

1.2 Question 2: Frequency Shifting (Modulation)

Problem Statement: Multiply the signal by a complex exponential $e^{jM\omega t}$ where $M = 5$. Compute the new coefficients and verify the Frequency Shifting property.

Theory & Intuition:

$$e^{jM\omega t} z(t) \leftrightarrow c_{n-M} \quad (2)$$

Intuition: Multiplying by $e^{jM\omega t}$ means you are spinning the entire drawing canvas at a constant angular velocity. Therefore, every epicycle has to spin M times faster to draw the same shape relative to the spinning canvas. The coefficient that used to be at index $n - M$ is now pushed to index n .

Code Solution:

```

M = 5
# 1. Create modulated signal
z_mod = z * np.exp(1j * M * fs_orig.omega * t)

# 2. Compute numerical coefficients
fs_mod = FourierEpicycles(t, z_mod, N_HARMONICS)
fs_mod.calculate_all_coefficients()

# 3. Verification
mse_list = []
for n in range(-N_HARMONICS + M, N_HARMONICS - M + 1):
    c_n_num = fs_mod.coeffs[n]
    c_n_theo = fs_orig.coeffs[n - M] # Shifted index
    mse_list.append(np.abs(c_n_num - c_n_theo)**2)

print(f"Freq Shift MSE: {np.mean(mse_list)}")

```

Code Explanation: Notice the loop bounds: `range(-N_HARMONICS + M, N_HARMONICS - M + 1)`. Because we shifted the spectrum by M , the theoretical coefficients at the extreme edges (e.g., $n = 50$) would require original coefficients that we didn't compute (e.g., $n = 45$, which we have, but $n = -50$ would need $n = -55$, which we don't have). We restrict the loop to valid indices.

2 Level 2: Harder (Calculus & Mixed Properties)

2.1 Question 3: Differentiation (Velocity of the Pen)

Problem Statement: Define $v(t) = \frac{d}{dt}z(t)$. Compute the Fourier coefficients of $v(t)$ numerically. Verify the differentiation property. Why might the MSE be higher for larger n ?

Theory & Intuition:

$$\frac{d}{dt}z(t) \leftrightarrow jn\omega c_n \quad (3)$$

Intuition: The derivative represents the velocity of the pen tracing the SVG. High-frequency epicycles (large n) spin very fast, contributing heavily to the velocity (sharp corners). Multiplying by j rotates the phase by 90° , which aligns with the fact that the derivative of sine is cosine.

Code Solution:

```
# 1. Compute numerical derivative
# dt is uniform, so we can use np.gradient
dt = t[1] - t[0]
v_t = np.gradient(z, dt)

# 2. Compute numerical coefficients
fs_diff = FourierEpicycles(t, v_t, N_HARMONICS)
fs_diff.calculate_all_coefficients()

# 3. Verification
mse_list = []
for n in range(-N_HARMONICS, N_HARMONICS + 1):
    c_n_num = fs_diff.coeffs[n]
    c_n_theo = 1j * n * fs_orig.omega * fs_orig.coeffs[n]
    mse_list.append(np.abs(c_n_num - c_n_theo)**2)

print(f"Differentiation MSE: {np.mean(mse_list)}")
```

Explanation & Alternatives: We use `np.gradient` for a second-order accurate central differences approximation of the derivative. *Alternative:* You could use finite differences manually: `v_t = np.diff(z) / dt`, but this reduces the array size by 1. You would have to append a value to keep the array size consistent with `t`. *Why is MSE higher for large n ?* Numerical differentiation amplifies noise and discretization errors. For high frequencies (large n), the discrete grid t does not have enough points to accurately capture the rapid changes, causing the numerical derivative to deviate from the theoretical continuous derivative.

2.2 Question 4: Time Reversal and Conjugation

Problem Statement: Create a signal $w(t) = z^*(-t)$. Verify its relationship to the original coefficients.

Theory & Intuition: Time Reversal: $z(-t) \leftrightarrow c_{-n}$ (Drawing the shape backwards swaps positive and negative frequencies). Conjugation: $z^*(t) \leftrightarrow c_n^*$ (Flipping the shape over the x-axis). Combined: $z^*(-t) \leftrightarrow (c_{-(-n)})^* = c_n^*$. *Intuition:* If you draw the shape backwards AND flip it upside down, the resulting epicycles are just the complex conjugates of the original ones.

Code Solution:

```
# 1. Time reversal and conjugation
# Time reversal of a periodic array: z(0) stays at 0, the rest is flipped.
z_rev = np.zeros_like(z)
z_rev[0] = z[0]
z_rev[1:] = z[1:][::-1]
w_t = np.conj(z_rev)

# 2. Compute coefficients
fs_w = FourierEpicycles(t, w_t, N_HARMONICS)
fs_w.calculate_all_coefficients()

# 3. Verification
mse_list = []
for n in range(-N_HARMONICS, N_HARMONICS + 1):
    c_n_num = fs_w.coeffs[n]
    c_n_theo = np.conj(fs_orig.coeffs[n])
    mse_list.append(np.abs(c_n_num - c_n_theo)**2)

print(f"Time Rev + Conj MSE: {np.mean(mse_list)}")
```

Code Explanation: Notice how time reversal is handled: `z[1:][::-1]`. Because $t = 0$ and $t = T$ are the same point in a periodic signal, reversing the array directly (`z[::-1]`) would shift the signal by one sample. Keeping index 0 fixed and reversing the rest is the mathematically correct way to do $z(-t)$ on a discrete periodic grid.

3 Level 3: Very Hard (Parseval's & Complex Combinations)

3.1 Question 5: Parseval's Relation (Energy Conservation)

Problem Statement: Verify Parseval's relation for the SVG drawing. Compute the total power in the time domain and compare it to the total power in the frequency domain.

Theory & Intuition:

$$P = \frac{1}{T} \int_0^T |z(t)|^2 dt = \sum_{n=-\infty}^{\infty} |c_n|^2 \quad (4)$$

Intuition: The total "energy" (or power) required to draw the shape in the time domain is exactly equal to the sum of the energies of all the individual rotating epicycles.

Code Solution:

```
# 1. Time Domain Power (Numerical Integration)
# |z(t)|^2 is the squared magnitude of the complex signal
power_time_integrand = np.abs(z)**2
power_time = np.trapezoid(power_time_integrand, t) / fs_orig.T

# 2. Frequency Domain Power (Sum of squared magnitudes)
# Note: We only have a finite N, so this will be an approximation.
power_freq = 0
for n in range(-N_HARMONICS, N_HARMONICS + 1):
    power_freq += np.abs(fs_orig.coefs[n])**2

print(f"Power (Time): {power_time}")
print(f"Power (Freq): {power_freq}")
print(f"Difference (Parseval Error): {np.abs(power_time - power_freq)}")
```

Explanation: The difference will not be exactly zero. Why? Because we truncated the Fourier series at $N = 50$. The time-domain integral contains the energy of *all* frequencies (up to infinity), while the frequency-domain sum only contains the energy up to $N = 50$. This difference is exactly the energy of the truncation error (Gibbs phenomenon).

3.2 Question 6: The "Funhouse Mirror" (Linearity + Scaling + Shift)

Problem Statement: Let $x(t)$ be the real part of $z(t)$ and $y(t)$ be the imaginary part. Create a new signal $w(t) = 2x(t - T/2) + j \cdot y(t)$. Compute its coefficients and verify against the theoretical prediction.

Theory & Intuition: First, express $x(t)$ and $y(t)$ in terms of $z(t)$: $x(t) = \frac{z(t) + z^*(t)}{2}$ and $y(t) = \frac{z(t) - z^*(t)}{2j}$. Therefore, $w(t) = 2 \left(\frac{z(t - T/2) + z^*(t - T/2)}{2} \right) + j \left(\frac{z(t) - z^*(t)}{2j} \right)$. $w(t) = z(t - T/2) + z^*(t - T/2) + 0.5z(t) - 0.5z^*(t)$. Using Linearity, Time Shift ($e^{-jn\pi} = (-1)^n$), and Conjugation (c_{-n}^*):

$$c_n^w = (-1)^n c_n + (-1)^n c_{-n}^* + 0.5c_n - 0.5c_{-n}^* \quad (5)$$

$$c_n^w = ((-1)^n + 0.5)c_n + ((-1)^n - 0.5)c_{-n}^* \quad (6)$$

Code Solution:

```
# 1. Create the signal
shift_idx = len(z) // 2
x_shifted = np.roll(z.real, shift_idx)
w_t = 2 * x_shifted + 1j * z.imag

# 2. Compute numerical coefficients
fs_w = FourierEpicyles(t, w_t, N_HARMONICS)
fs_w.calculate_all_coefficients()

# 3. Verification
mse_list = []
for n in range(-N_HARMONICS, N_HARMONICS + 1):
    c_n_num = fs_w.coefs[n]

    c_n = fs_orig.coefs[n]
    c_minus_n_conj = np.conj(fs_orig.coefs[-n])

    term1 = ((-1)**n + 0.5) * c_n
    term2 = ((-1)**n - 0.5) * c_minus_n_conj
    c_n_theo = term1 + term2
```

```
mse_list.append(np.abs(c_n_num - c_n_theo)**2)
print(f"Funhouse Mirror MSE: {np.mean(mse_list)}")
```

Explanation: This is the ultimate test of understanding. It forces you to break down a complex spatial manipulation into its fundamental mathematical properties (Linearity, Shift, Conjugation) and apply them simultaneously. If you can solve this, you can solve any 1D Fourier Series question they throw at you.